第2回：JDBC接続の基本〜JavaからDBへ「橋」をかける〜

1. ① 概要：接続 → 準備 → DB へのアクセス → クローズ

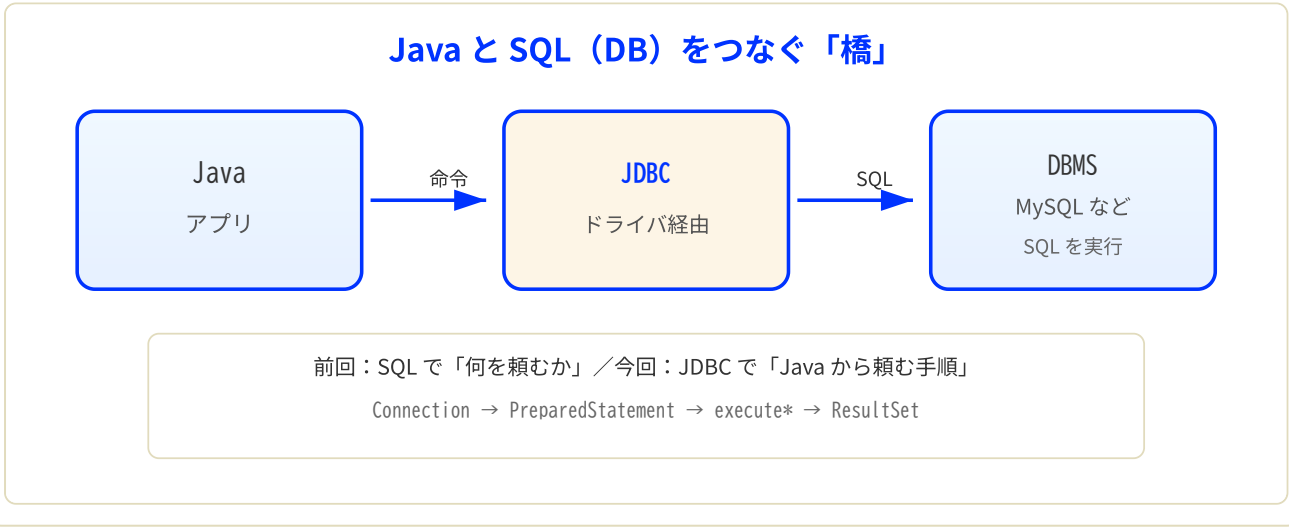
今日は、**Java から DB に命令を出すための入口（JDBC）**を、短い手順として固める回です。

この回の**全体の流れ（型）**は次のとおりです。細部はまだ追わず、**順番だけ**頭に入れてください。

```
Connection（接続）→ PreparedStatement（SQL 準備・? へのバインド）
→ executeQuery / executeUpdate（DB へのアクセス＝実行）
→ ResultSet（SELECT のとき結果を読む）／更新件数を受け取る
→ try-with-resources または finally でリソースをクローズ
```

SELECT なら `executeQuery`、INSERT/UPDATE/DELETE なら `executeUpdate` と覚えておくと、後の図・コードと対応しやすくなります。

Java と SQL（DB）の位置づけ：SQL は「言語」、JDBC は「Java からその言語を運ぶための橋（API）」です。



2. ② 工程を追う（学習目標・用語・図）

2-1. 学習目標（手順を覚える）

この回では、JDBC の基本パイプラインを**順番通りに言える**こと、と**安全側の書き方（? とバインド）**を掴むことを目指します。

▼ JDBC の4ステップを順番で説明できる

接続（Connection）→ SQL 準備（PreparedStatement）→ 実行（execute*）→ 結果（ResultSet）の流れを、図とあわせて説明できる。

- **Connection**：DB への接続を作る。
- **PreparedStatement**：SQL を安全に組み立てる（? に値をバインド）。
- **executeQuery / executeUpdate**：SELECT と更新系で使い分ける。
- **ResultSet**：SELECT の結果を1行ずつ読む。

▼ PreparedStatement を使う理由を一言で言える

値を文字列連結で SQL に埋め込まず、**? に後から値を渡す**ことで、SQL インジェクション（悪意ある入力の混入）を避けやすくする、という説明ができる。

▼ 手順の型（写経テンプレ）

次の並びを、自分の言葉でも追えること。

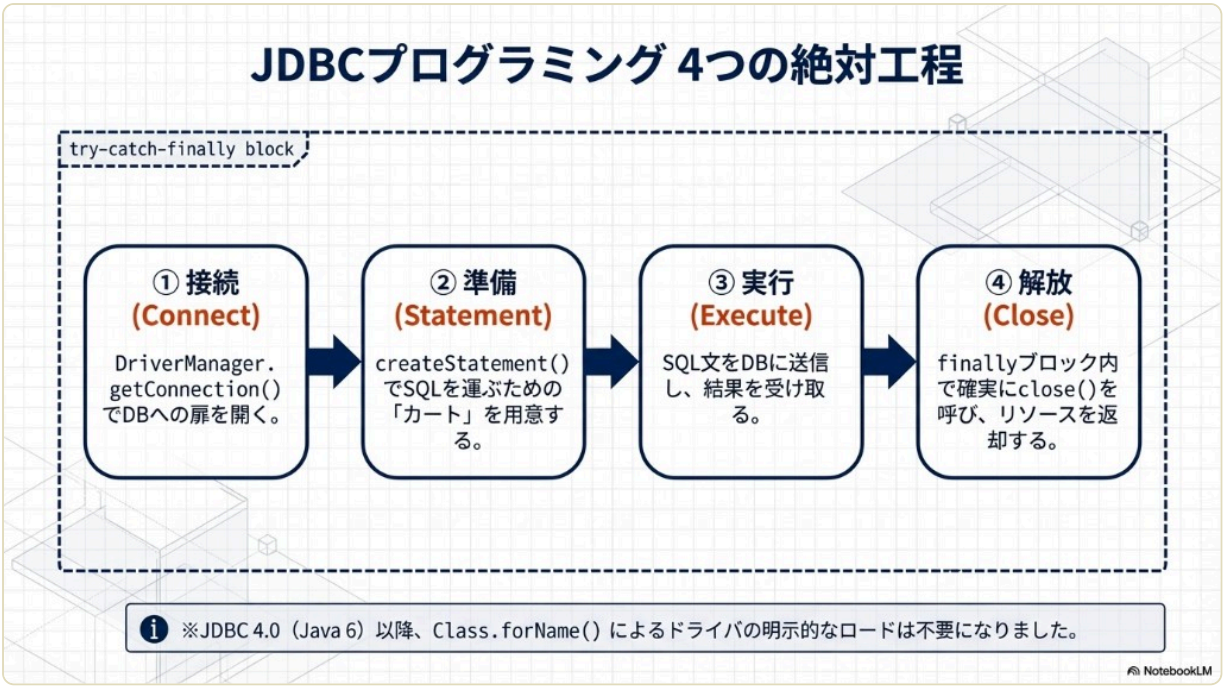
```
Connection ▶ PreparedStatement ▶ executeQuery / executeUpdate ▶ ResultSet
```

2-2. 用語の確認（最小）

- **JDBC**：Java から DB にアクセスするための標準 API（ドライバ経由で DBMS と会話する）。
- **Connection**：DB への接続（セッション）を表すオブジェクト。
- **PreparedStatement**：SQL を一度形にし、`?` に値を後から埋めるためのオブジェクト。
- **ResultSet**：SELECT の結果（表のイメージ）を、カーソルを進めながら読むオブジェクト。
- **ドライバ（mysql-connector-j など）**：JDBC の命令を、各 DBMS が理解できる形に変換する部品（クラスパスに追加する）。

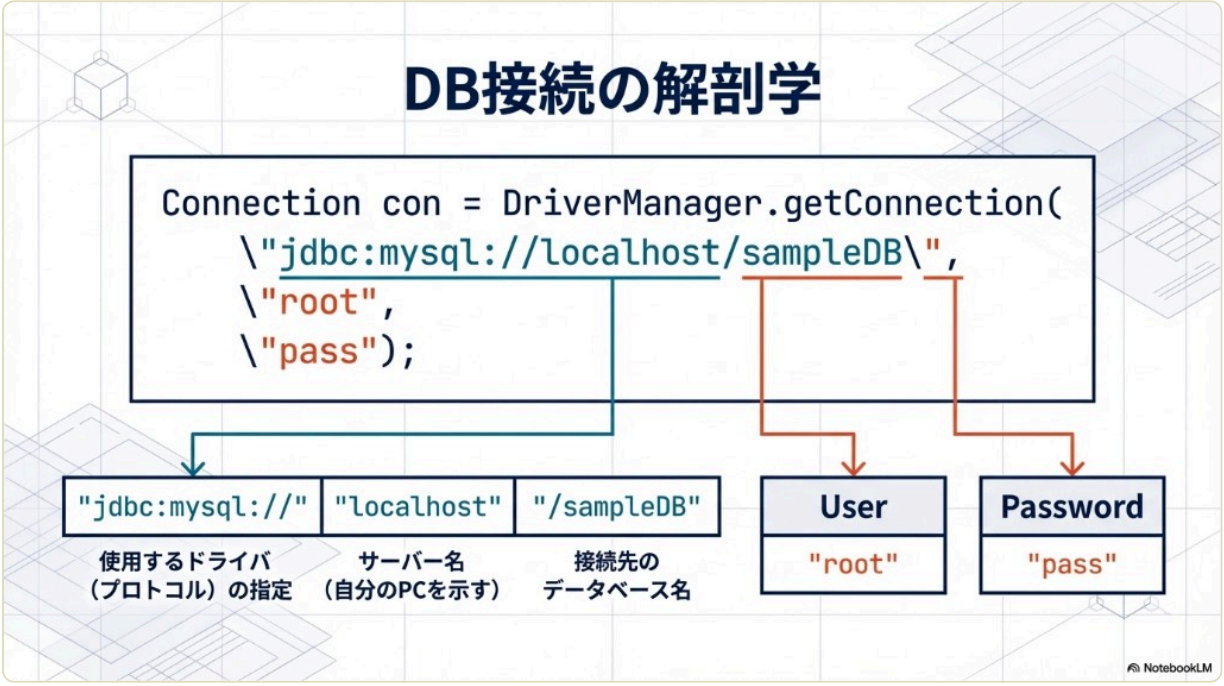
2-3. 図でつかむ（4ステップ・安全性・読み方）

4ステップ全体像



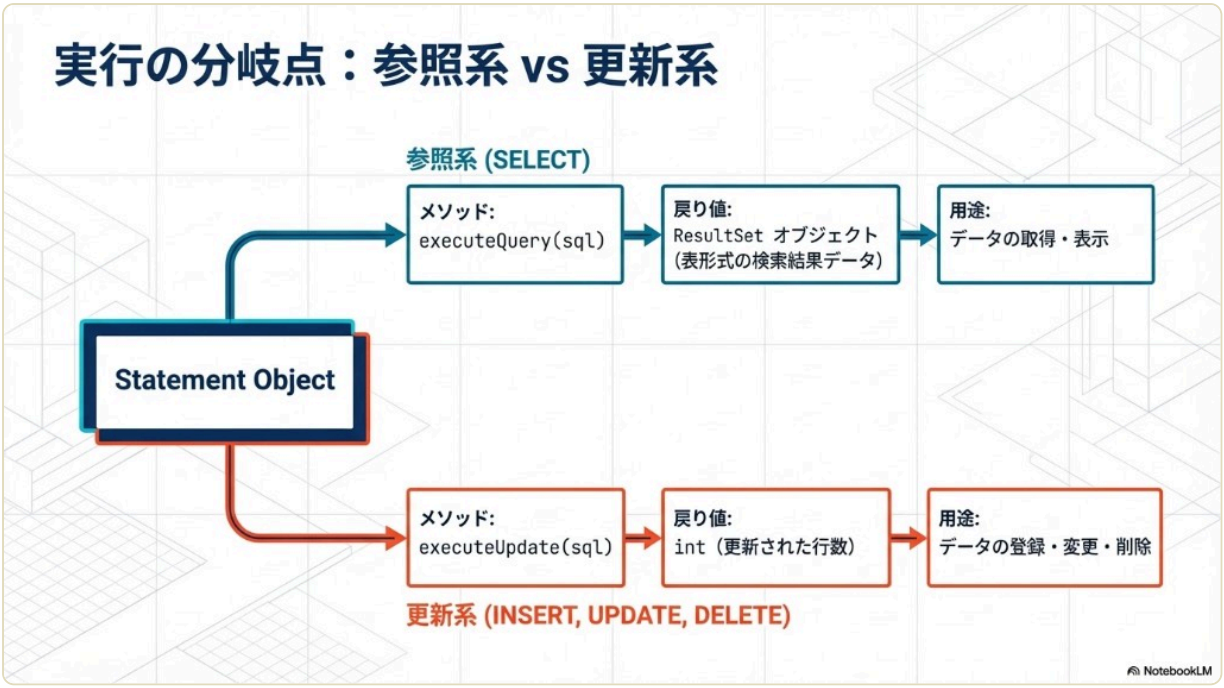
JDBC は「接続 → 準備 → 実行 → 解放」をこの順番で回す、という全体像をまずイメージしてください。

DB 接続 URL の分解



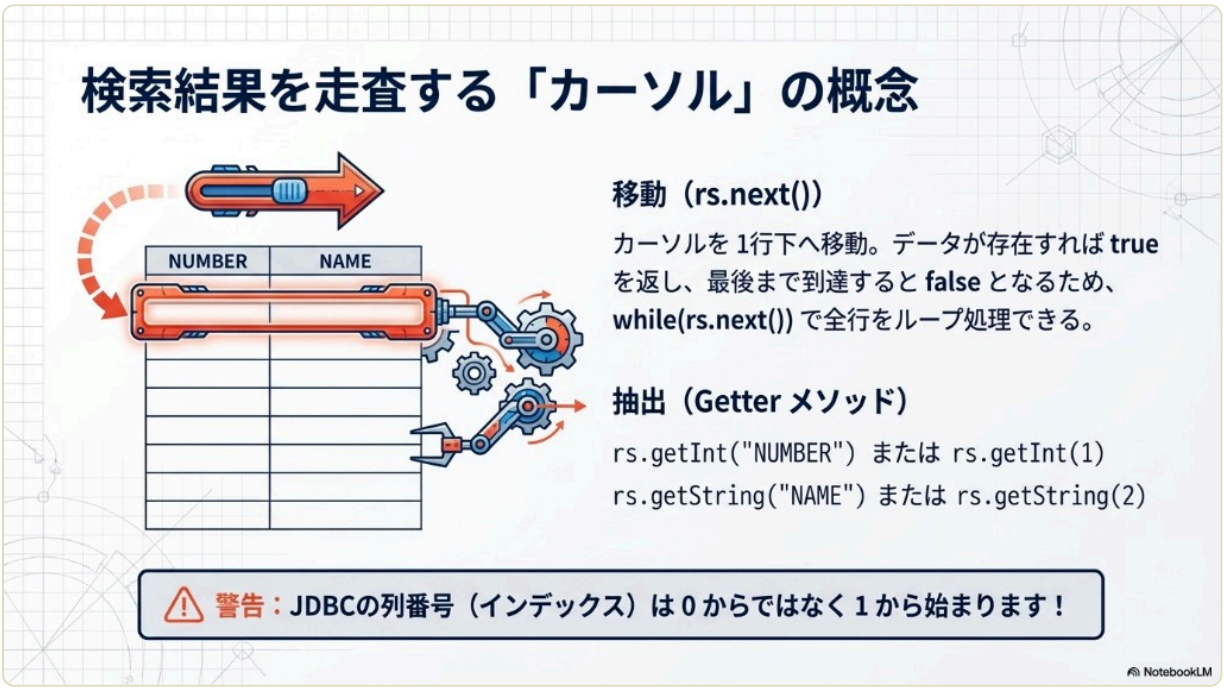
URL の中で「どのDBに、どのユーザでつなぐか」を一つずつ分解して押さえましょう。

実行メソッドの分岐



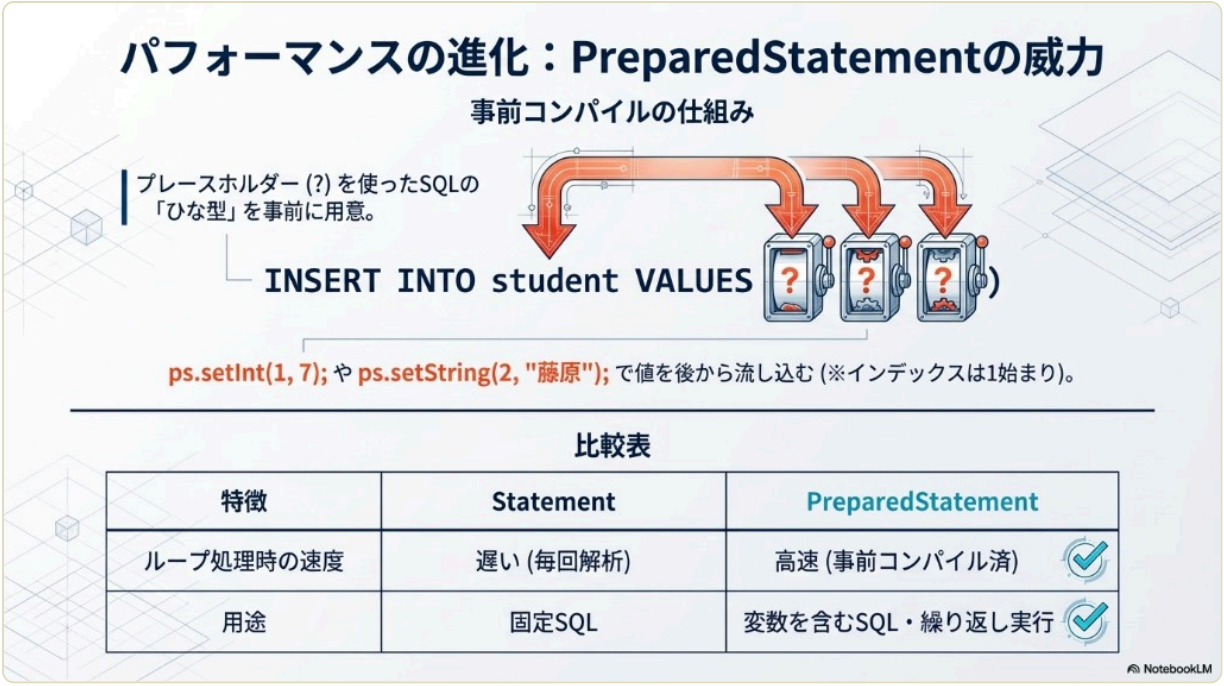
SELECT で結果セットを返すときは `executeQuery`、INSERT/UPDATE/DELETE で更新件数だけほしいときは `executeUpdate` を使います。

ResultSet のカーソル



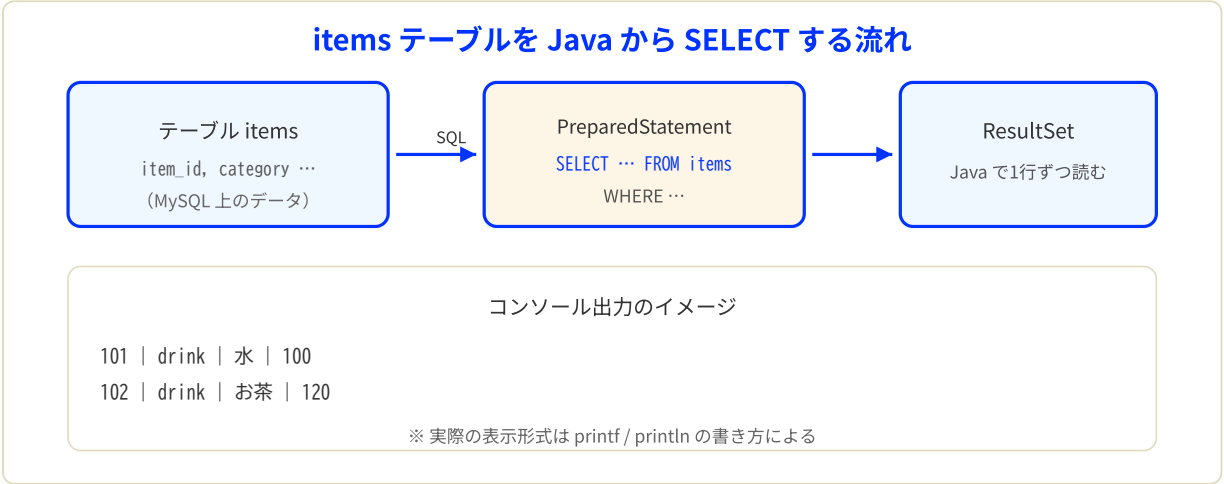
カーソルが `rs.next()` で1行ずつ進み、各行の値を `getInt` や `getString` で取り出すイメージです。

PreparedStatement と？（プレースホルダ）



SQL の「ひな型」を先に用意し、あとから `setString` などで値を流し込むことで、安全かつ高速に実行しやすくなります。

items テーブルを Java から SELECT（流れの総まとめ）



ここまでの 4ステップ・URL・実行メソッド・ResultSet・PreparedStatement を、`items` テーブルの具体例で一度つなげて眺めてみましょう。

3. ③ ソースコードの説明

3-1. 参考：try-catch-finally と手動 close（昔の書き方）

JDBC は `Connection` / `PreparedStatement` / `ResultSet` を必ず閉じる必要があります。昔は **finally で close を書く**形が定番でした。

ただし、この形はコードが長くなりやすいので、**Java 7 以降は try-with-resources（次の節の形）**を基本にします。**この例では説明を簡単にするため、close は Connection（con）だけ**を示します。

```
import java.sql.*;

public class LegacyCloseSample {
        public static void main(String[] args) {
                Connection con = null;
                PreparedStatement ps = null;
                ResultSet rs = null;

                try {
                        con = DriverManager.getConnection("jdbc:mysql://localhost:3306/s
                        ps = con.prepareStatement("SELECT item_id FROM items WHERE categ
                        ps.setString(1, "drink");
                        rs = ps.executeQuery();
                        while (rs.next()) {
                                System.out.println(rs.getInt("item_id"));
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                } finally {
                        if (con != null) {
                                try {
                                        con.close();
                                } catch (SQLException e) {
                                        e.printStackTrace();
                                }
                        }
                }
        }
}
```

▼ レガシーな finally/close の例をドラッグでコピー（クリックで開く）

下のプレーンテキストの貼り付け欄から、Eclipse などへドラッグで取り込めます。

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.*;

public class LegacyCloseSample {
        public static void main(String[] args) {
                Connection con = null;
                PreparedStatement ps = null;
                ResultSet rs = null;
```

3-2. Java サンプル（items：category を ? で検索）

次のサンプルは、**接続情報（URL / ユーザー / パスワード）**は自分の環境に合わせて置き換えてください。DB 側に `items` がある前提です（前回の DDL を流し込んでおくとスムーズです）。

依存ライブラリ（例：**mysql-connector-j**）はプロジェクトに追加し、実行時のクラスパスに含めます。

> **先に環境構築**：JAR のDL（learningBOX）→ Eclipse の外部JAR追加 → 接続確認（SampleConnection）までを先に済ませてください。環境構築ページ

サンプル1：SELECT（executeQuery）

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ItemsSelectSample {

        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverT
                String user = "root";
                String password = "password";

                String sql = "SELECT item_id, category, item_name, unit_price FROM items

                try (
                        Connection conn = DriverManager.getConnection(url, user, passwor
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");

                        try (ResultSet rs = ps.executeQuery()) {
                                while (rs.next()) {
                                        int itemId = rs.getInt("item_id");
                                        String category = rs.getString("category");
                                        String itemName = rs.getString("item_name");
                                        int unitPrice = rs.getInt("unit_price");

                                        System.out.println(itemId + " | " + category + "
                                }
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ サンプルコードをドラッグでコピー（クリックで開く）

下のプレーンテキストの貼り付け欄から、Eclipse などへドラッグで取り込めます。

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ItemsSelectSample {
```

サンプル2：INSERT（executeUpdate）

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ItemsInsertSample {

        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverT
                String user = "root";
                String password = "password";

                String sql = "INSERT INTO items (category, item_name, unit_price) VALUES

                try (
                        Connection conn = DriverManager.getConnection(url, user, passwor
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");
                        ps.setString(2, "sample juice");
                        ps.setInt(3, 120);

                        int updated = ps.executeUpdate();
                        System.out.println(updated + " 件追加しました");
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ **サンプル2（INSERT）のコードをドラッグでコピー（クリックで開く）**

下のプレーンテキストの貼り付け欄から、Eclipse などへドラッグで取り込めます。

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.PreparedStatement;
import java.sql.SQLException;

public class ItemsInsertSample {

        public static void main(String[] args) {
```

3-3. サンプルと対応付け（JDBC URL・Setter・Getter・カーソル）

上の **サンプル1（SELECT）** の各行が、② の図のどこに当たるかを対応づけます。

- **JDBC URL（`String url`）**：`jdbc:mysql://ホスト:ポート/DB名` に続く `?` 以降は接続オプション（SSL やタイムゾーンなど）。`user` / `password` は `getConnection` の第2・第3引数で渡します。
- **Setter（バインド）**：SQL 中の `?` に、`setString(番号, 値)` や `setInt` で値を安全に流し込みます。番号は SQL 上の `?` の左から 1, 2, … です。
- **Getter（結果の取り出し）**：`ResultSet` に対して `getInt("列名")` / `getString("列名")` などで、現在の行のセルを Java の型で取り出します。
- **カーソル**：`ResultSet` は「表」全体ではなく、**今どの行を指しているか**というカーソルを持ちます。初期位置はデータ行の手前で、`next()` が `true` のあいだ、1行ずつ進めながら getter で読みます。

4. ④ 写経・練習問題

② の図・用語、③ のサンプルで押さえた**同じ流れ**が、次の問題では「言葉」（4-1）と「コード上の空欄」（4-2）のどちらでも追えるようになっています。手元のメモ用紙か IDE で答え、まず自分で考えてからヒント・解答を開いてください。

4-1. 用語・手順の確認（問題1〜5）

問題1（選択式）：次の選択肢 A〜D から、JDBC の基本パイプライン（4ステップ）を**正しい順番**で並べているものを1つ選びなさい。

ここでいう「4ステップ」は**オブジェクトの並びとしての型**です。SELECT では `executeQuery`、更新系では `executeUpdate` を使う、と③までで押さえたうえで、選択肢の中から**順序として正しいもの**を選んでください。

選択肢（この中の語を使います）

`Connection` / `PreparedStatement` / `executeQuery` / `executeUpdate` / `ResultSet`

```
A) Connection → PreparedStatement → executeQuery / executeUpdate → ResultSet
B) PreparedStatement → Connection → executeQuery / executeUpdate → ResultSet
C) Connection → executeQuery / executeUpdate → PreparedStatement → ResultSet
D) Connection → PreparedStatement → ResultSet → executeQuery / executeUpdate
```

▼ ヒント

「つなぐ → SQL準備 → 実行 → 結果を読む」の順です。SELECT の結果を最後に扱うのはどのオブジェクトでしたか。

▼ 解答例

```
正解：A

Connection → PreparedStatement → executeQuery / executeUpdate → ResultSet
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
正解：A

Connection → PreparedStatement → executeQuery / executeUpdate → ResultSet
```

問題2：`SELECT` を実行するときに使うメソッド名を書いてください。

▼ ヒント

結果セット（表のイメージ）を返すメソッドです。

▼ 解答例

```
executeQuery()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
executeQuery()
```

問題3： `INSERT` / `UPDATE` / `DELETE` を実行するときに使うメソッド名を書いてください。

▼ ヒント

更新件数（int）を返す系の実行メソッドです。

▼ 解答例

```
executeUpdate()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
executeUpdate()
```

問題4： `ResultSet` で「次の行」に進むメソッド名を書いてください。

▼ ヒント

`while` ループとセットでよく書きます。

▼ 解答例

```
next()
```

マウスで全選択して貼り付け欄へドラッグしてください：

```
next()
```

問題5：`?` に値を入れる処理を、教科書的な用語で何と呼ぶか書いてください（一般的な日本語名でOK）。

▼ ヒント

`setString` / `setInt` などの前段の話です。

▼ 解答例

バインド（パラメータバインド）

マウスで全選択して貼り付け欄へドラッグしてください：

バインド（パラメータバインド）

4-2. コード読解（穴埋め：問題7〜10）

③ のサンプルと**同じ形のクラス全体**を載せています（**import 行は載せません**。手元のプロジェクトで `java.sql.*` などを追加してください）。`/* ??? */` に入る**型名・メソッド名**を答えます。

問題7：コード読解（SELECT・基本） — 次のコードの `/* ??? */` に入る適切な語（型名）を書いてください。

```
// import は手元で追加（例: java.sql.* および SQLException）

public class CodeRead07 {
        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverTimez
                String user = "root";
                String password = "password";

                String sql = "SELECT item_id, category, item_name, unit_price FROM items WHE

                try (
                        // ① DB に接続するオブジェクトの型
                        /* ??? */ conn = DriverManager.getConnection(url, user, password);
                        // ② SQL を準備するオブジェクトの型
                        /* ??? */ ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");

                        // ③ SELECT の結果を受け取るオブジェクトの型（executeQuery の戻り値
                        try (/* ??? */ rs = ps.executeQuery()) {
                                while (rs.next()) {
                                        int itemId = rs.getInt("item_id");
                                        String category = rs.getString("category");
                                        String itemName = rs.getString("item_name");
                                        int unitPrice = rs.getInt("unit_price");

                                        System.out.println(itemId + " | " + category + " | "
                                }
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ **ヒント**

4ステップのうち、「接続」「準備」「結果セット」を表すクラス名と、SELECT に使う実行メソッド名を思い出してください。

▼ 解答例

```
/* ??? */ conn → Connection
/* ??? */ ps   → PreparedStatement
/* ??? */ rs   → ResultSet（executeQuery の戻り値）
```

問題8：コード読解（SELECT・カーソル） — 問題7と同じ前提の**クラス全体**のうち、`/* ??? */` に入る**メソッド名**だけを書いてください。

```
// import は手元で追加（例: java.sql.* および SQLException）

public class CodeRead08 {
        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverTimez
                String user = "root";
                String password = "password";
                String sql = "SELECT item_id FROM items WHERE category = ? ORDER BY item_id"

                try (
                        Connection conn = DriverManager.getConnection(url, user, password);
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");

                        try (ResultSet rs = ps.executeQuery()) {
                                // ① 次の行に進むメソッド
                                while (rs. /* ??? */ ()) {
                                        // ② 列 item_id を int で取り出すメソッド
                                        int itemId = rs. /* ??? */ ("item_id");
                                        System.out.println(itemId);
                                }
                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ ヒント

`ResultSet` のカーソルを動かすメソッドと、int 型を取り出す getter メソッドです。

▼ 解答例

```
while (rs. /* ??? */ ()) → rs.next()
int itemId = rs. /* ??? */ ("item_id"); → rs.getInt("item_id")
```

問題9：コード読解（UPDATE・基本） — 次のコードの `/* ??? */` に入る適切な**メソッド名**を書いてください。

```
// import は手元で追加（例: java.sql.* および SQLException）

public class CodeRead09 {
        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverTimez
                String user = "root";
                String password = "password";
                String sql = "UPDATE items SET unit_price = ? WHERE item_id = ?";

                try (
                        Connection conn = DriverManager.getConnection(url, user, password);
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setInt(1, 150);
                        ps.setInt(2, 1);

                        // ① 更新系で使う実行メソッド
                        int updated = ps. /* ??? */ ();
                        System.out.println(updated + " 件更新しました");
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ ヒント

INSERT/UPDATE/DELETE で更新件数（int）を返すメソッドです。

▼ 解答例

```
ps. /* ??? */ (); → ps.executeUpdate();
```

問題10：コード読解（INSERT・バインド） — 次のコードの `/* ??? */` に入る適切な語（型名・メソッド名）を書いてください。

```
// import は手元で追加（例: java.sql.* および SQLException）

public class CodeRead10 {
        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverTimez
                String user = "root";
                String password = "password";
                String sql = "INSERT INTO items (category, item_name, unit_price) VALUES (?,

                try (
                        /* ??? */ conn = DriverManager.getConnection(url, user, password);
                        /* ??? */ ps = conn.prepareStatement(sql);
                ) {
                        // ① 1つ目の ? に文字列を入れるメソッド
                        ps. /* ??? */ (1, "drink");
                        ps.setString(2, "sample juice");
                        // ② 3つ目の ? に int を入れるメソッド
                        ps. /* ??? */ (3, 120);

                        int updated = ps.executeUpdate();
                        System.out.println(updated + " 件追加しました");
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ ヒント

`Connection` / `PreparedStatement` / `setString` / `setInt` から選んでください。

▼ 解答例

```
/* ??? */ conn → Connection
/* ??? */ ps   → PreparedStatement
ps. /* ??? */ (1, "drink"); → ps.setString(1, "drink");
ps. /* ??? */ (3, 120);     → ps.setInt(3, 120);
```

4-3. ラボ（10分チェックリスト）

10分で終わる最小確認（講師のOKが出るまで次に進めない、の運用にしてください）

1. JDBC URL（ホスト名・ポート・DB名）が自分の環境と一致している。
2. MySQL ドライバ（例：mysql-connector-j）がプロジェクトに入り、実行できる。
3. `SELECT` が `executeQuery` で1回通り、コンソールに行が出る。
4. `?` のバインド（例：`setString(1, "drink")`）を1か所変えて、結果が変わることを確認する。
5. 例外が出たら、スタックトレースの先頭メッセージを読み、接続／SQL／権限のどれかに切り分ける。
6. 余力がある人だけ：`INSERT` を `executeUpdate` で1回通し、次の `SELECT` で増えたことを確認する。

5. ⑤ 発展問題（余力向け）

④ の基本が一通りできたあとに取り組む想定です。ヒント・解答は折りたたみにまとめています。

発展1（記述）：ユーザー入力を文字列連結で SQL に埋め込まず、`PreparedStatement` を使う主な理由を、セキュリティ観点で一言で書いてください。

▼ ヒント

「入力が SQL の形を壊して、意図しない命令が混ざる」系の攻撃を、設計上しづらくします。

▼ 解答例

SQL インジェクション対策（悪意ある入力を SQL として解釈されにくくする）

マウスで全選択して貼り付け欄へドラッグしてください：

SQL インジェクション対策（悪意ある入力を SQL として解釈されにくくする）

発展2（コード）：次のプログラムのうち、`ResultSet` を開いたあとのブロックが空です。**コメントの指示どおり**、`while` でカーソルを進めながら `item_id` と `item_name` を1行ずつ表示するコードを書いてください（import は手元で追加）。

```
// import は手元で追加（例: java.sql.* および SQLException）

public class CodeAdvanced02 {
        public static void main(String[] args) {
                String url = "jdbc:mysql://localhost:3306/sample_db?useSSL=false&serverTimez
                String user = "root";
                String password = "password";
                String sql = "SELECT item_id, item_name FROM items WHERE category = ? ORDER

                try (
                        Connection conn = DriverManager.getConnection(url, user, password);
                        PreparedStatement ps = conn.prepareStatement(sql);
                ) {
                        ps.setString(1, "drink");

                        try (ResultSet rs = ps.executeQuery()) {
                                // ここで while を使って、カーソルを移動しつつ item_id と it

                        }
                } catch (SQLException e) {
                        e.printStackTrace();
                }
        }
}
```

▼ ヒント

`rs.next()` が `true` のあいだ繰り返し、`getInt` / `getString` で列を取り出します。

▼ 解答例

```
try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
                int id = rs.getInt("item_id");
                String name = rs.getString("item_name");
                System.out.println(id + " | " + name);
        }
}
```

6. まとめ（①の型を思い出す）

最後に、① で示した JDBC の流れを短く復習します。

```
1) Connection：DB に接続する
2) PreparedStatement：SQL を準備し、? に値をバインドする
3) 実行：SELECT は executeQuery／更新は executeUpdate
4) ResultSet：SELECT の結果を next と get* で読む
```